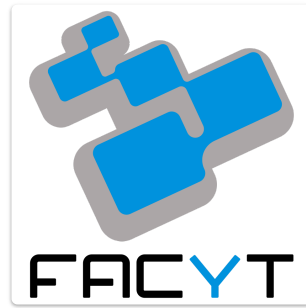




Sistemas Operativos

Asignación N°1: Repaso de Arquitectura del Computador

ESTUDIANTE: Jesús Hernández

CÉDULA: 31.456.568

FECHA: Mayo, 2026

Resumen

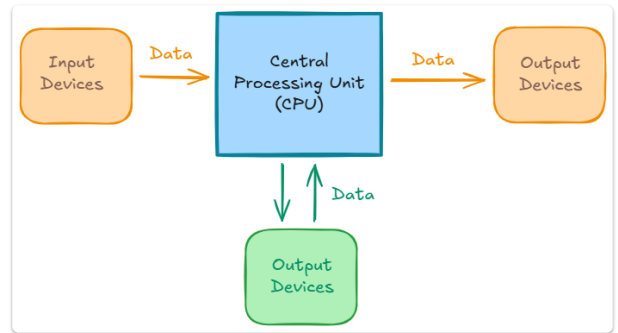
El presente informe técnico aborda el repaso fundamental de la Arquitectura del Computador como base indispensable para la comprensión de los Sistemas Operativos modernos. En primer lugar, se analiza la organización estructural del hardware y software, detallando los elementos fundamentales de las instrucciones y analizando minuciosamente las fases del ciclo de instrucción clásico controlado por el procesador. Posteriormente, se profundiza en el mecanismo de gestión de interrupciones, evaluando su importancia crucial en el diseño de sistemas de tiempo compartido, sus clasificaciones, el enmascaramiento y los esquemas de arbitraje síncrono y asíncrono ante solicitudes concurrentes. Finalmente, el documento examina tópicos avanzados en la estructura del Kernel de Linux, contemplando las optimizaciones estructurales en la línea reciente de versiones 7.0, el impacto de seguridad y rendimiento derivado de la incorporación del lenguaje Rust en el núcleo monolítico, y un análisis comparativo y taxonómico sobre tecnologías de virtualización e hipervisores frente al aislamiento de contenedores.

Repaso de Arquitectura del Computador

Organización del computador

Los ordenadores digitales, dispositivos finitos y discretos que almacenan y manipulan el estado de valores booleanos en mapas complejos, están compuestos por hardware (el equipo físico) y software (instrucciones que se fabrican para operar el equipo). Todo está interconectado por un sistema de buses (datos, direcciones y control).

El hardware informático está compuesto de dispositivos electrónicos o electromecánicos con:

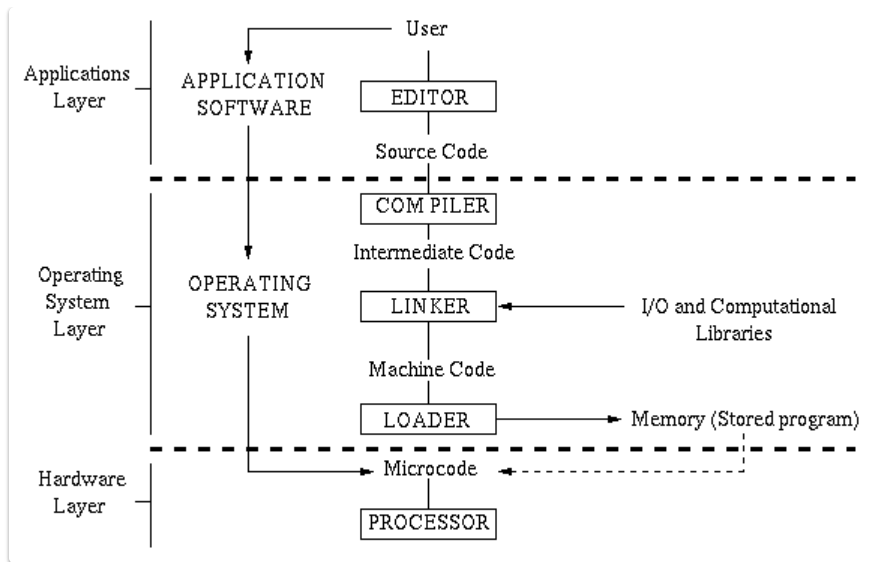


- **La Memoria (RAM y Cache):** un conjunto de registros y dispositivos de almacenamiento de información booleana que almacenan el mapa de procesos del equipo. Es un espacio volátil y direccionable que retienen también instrucciones y datos activos. Es de acceso más rápido que los discos duros.
- **La Unidad Central de Procesamiento (CPU):** es el objeto que realiza el trabajo principal del equipo. Cambia los estados del mapa de estados que está en la memoria. Está compuesto por la Unidad de Control (UC) que codifica y coordina el flujo de datos, la Unidad Aritmético-Lógica (ALU) que ejecuta operaciones binarias, y los registros internos de alta velocidad.
- **Procesador de Entrada/Salida (E/S):** gestiona y realiza el trabajo asociado a la lectura y escritura de la información que se añade, se resta o se copia a una parte del mapa de procesos del computador.
- **Periféricos:** incluyen dispositivos que almacenan software y datos auxiliares, dispositivos de entrada como ratón y teclado, dispositivos de salida como impresoras o dispositivos de visualización como las pantallas y monitores.

En el software informático se incluye conjuntos de instrucciones *programas* tales como:

- **Microcódigo:** instrucciones de bajo nivel que hacen funcionar a la CPU.
- **Código Máquina:** código en un nivel ligeramente superior al microcódigo, estas instrucciones se cargan directamente en la memoria y comprenden un *programa almacenado ejecutable*.
- **Librerías Computacionales y de E/S:** Son sets de procedimientos e instrucciones que el procesador de E/S y la CPU utilizan para importar y exportar datos, así como calcular diversas operaciones como aritmética, matemática y manejo de datos.
- **Software de aplicaciones:** Son los programas que los usuarios corren en una máquina para realizar diversas actividades en casa, actividades, negocios y tareas científicas. Por ejemplo:
 - Procesadores de texto (como Microsoft Word, Visual Studio Code),
 - Hojas de cálculo (como Excel),
 - Dibujo o boceto (AutoCAD, Asesprite, Paint),
 - Gráficos (Adobe Photoshop),
 - Video (Vegas Pro),
 - Aplicaciones que procesan datos como las aplicaciones de machine learning o procesadores de lenguaje natural (como Deepseek),
 - Conectarse a internet (como Google Chrome) o incluso
 - Videojuegos (como Octopath Traveler).
 - Entre otros.
- **Sistema Operativo:** Aplicaciones de software de interfaces con librerías y microcódigo (en muy pocos casos) de manera conveniente y transparente (es decir, invisible) para el usuario.
- **Servicios Auxiliares:** Además de las utilidades proporcionadas por el sistema operativo del ordenador, puede haber servicios auxiliares para los usuarios como compiladores, enlazadores y cargadores para lenguajes de programación; como Pascal, C o C++.

Organización del software informático para compilación y la ejecución de software de aplicaciones. Las líneas sólidas y ligeras denotan flujo de datos, mientras que las líneas punteadas ligeras denotan el flujo de control.



Instrucciones

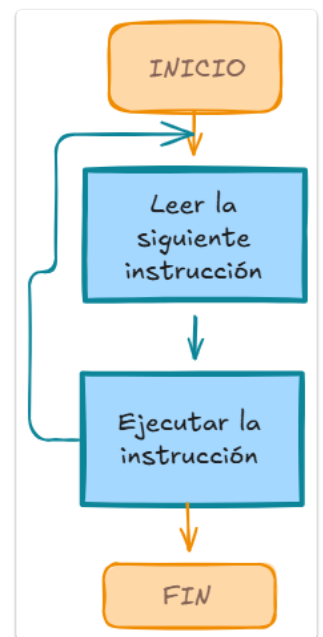
El principal propósito de un computador es ejecutar programas y solucionar problemas. Para eso se necesitan instrucciones, poder decirle procesador que tiene que hacer en cada instante o el orden de acciones a realizar en un determinado tiempo; la operación elemental. La ejecución de un problema consiste en la repetición de un proceso llamado *ciclo de la instrucción*; el problema trae la instrucción desde la memoria principal y las va ejecutando progresivamente.

Una instrucción está en una forma de código binario, donde se especifica cual es la acción que el procesador llevará a cabo, el procesador solo se limita a interpretar la instrucción y hacerla. Entre las acciones que puede realizar una instrucción en un programa se listan las siguientes.

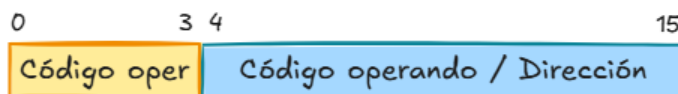
- **Procesador-Memoria:** Se transfieren datos del procesador a la memoria o viceversa.
- **Procesador de E/S:** Se transfieren datos desde o hacia un dispositivo.
- **Tratamiento de datos:** El procesador realiza alguna operación aritmética o lógica sobre los datos.
- **Control:** La instrucción pide que se altere la secuencia de ejecución.

Las instrucciones poseen dos campos:

- **El código de operación (Opcode),** que representa la acción que el procesador debe ejecutar. El número de bits asignado al determina cuántas instrucciones distintas puede tener el procesador (2^n instrucciones, donde n es el número de bits).
- **El código operando (Direcciones/Registros),** que define los parámetros de la acción. El código operando depende a su vez de la operación y su longitud es el resto de los bits no usados por el Opcode. Puede tratarse de direcciones de memoria de origen o destino, número de registros de la CPU o datos inmediatos (valores constantes).



Ejemplo de instrucción:



Ejemplo de entero:



El **tamaño de la instrucción** depende de la arquitectura de la máquina, puede tener un tamaño **fijo** (arquitecturas RISC como ARM) o de tamaño **variable** (arquitecturas CISC como x86). El tamaño de la instrucción determina el espacio físico disponible para repartir entre la

operación y los datos a manipular. Un mayor tamaño permite direccionar más memoria y ofrecer más funciones, pero requiere un bus de datos más ancho y más ciclos de lectura.

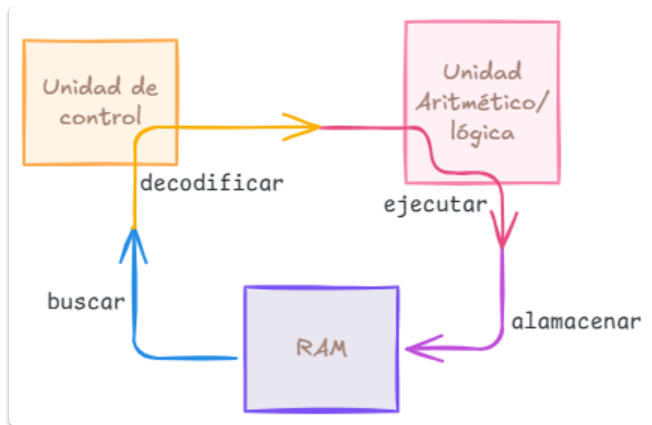
Tipos de instrucciones

Como la longitud del Opcode de una instrucción determina cuantas operaciones/instrucciones se pueden realizar, las arquitecturas definen conjuntos de operaciones distintas en función de su objetivo.

Tipo de instrucción	Nota	Ejemplos en x86
Transferencia de Datos	Pasos: 1. Determinación de direcciones de memoria de origen y destino 2. Realización de la transformación de memoria virtual a memoria real. 3. Comprobación de caché. 4. Inicio del proceso de lectura/escritura en la memoria.	- LDS (cargar DS) - MOV (mover) - LEA (cargar dirección efectiva) - LES (cargar LES) - XCHG (intercambio)
Aritméticas	Las más sencillas como sumas y restas, pero con ayuda del co-procesador se pueden realizar operaciones más complejas.	- ADD (sumar) - ADC (sumar con acarreo) - SUB (resta) - SBB (resta con acarreo) - CBW (convierte bytes en words) - CWD (convierte un word a doble word). - CMP (compara). - INC (incrementa). - DEC (decrementa).
Lógicas	Son utilizadas en operaciones bits a bits. Incluyen operaciones de comparación lógica.	- OR - AND - XOR - TEST - NOT
De Salto	Sirven para que el procesador, en lugar de ejecutar la siguiente instrucción, pase a ejecutar otra en una posición denominada "destino de salto".	- JZ (salta si es cero o igual) - JNZ (salta si no es cero o es igual) - JS (salta si la bandera de signo está activa) - JNS (salta si la bandera de signo no está activa) - JO (salta si hay desbordamiento) - JNO (salta si no hay desbordamiento)
Instrucciones de E/S	Administran los comandos de entrada/salida. Si hay un mapa de memoria de E/S, determina la dirección de este mapa.	- IN - OUT - INS - OUTS
Instrucciones de control de sistema	Son aquellas instrucciones que se ejecutan solo cuando el procesador esta en un estado de privilegio o ejecuta un programa que esta en una zona privilegiada de memoria.	- HLT - CLI - STI - LGDT - LIDT

Procesamiento de una instrucción

Dentro de un procesador, en cada **ciclo de CPU** o **ciclo de instrucción** se procesa una instrucción. Este proceso es continuo y secuencial, repetido miles de millones de veces por segundo según el equipo, y su conjunto permite ejecutar un programa o una aplicación.



Las cuatro fases fundamentales de este proceso son:

1. **Búsqueda (fetch):** La CPU localiza y extrae la instrucción desde la memoria principal (RAM). Es dentro donde se encuentra la información que la CPU debe procesar.
 - El **program counter (PC)** apunta a la siguiente línea de memoria donde se encuentra la siguiente instrucción del procesador. Se incrementa en 1 su valor cada vez que se termina un ciclo completo de instrucción o cuando una instrucción de salto cambia el valor del contador de programa.
 - En el **registro de direccionamiento de memoria o Memory Address Register (MAR)** se copia el contenido del PC y se envía a la RAM a través de los pines de direccionamiento de la CPU, los cuales están cableados con los pines de direccionamiento de la propia RAM.
 - En el caso de que la CPU tenga que realizar una lectura de memoria, el **registro de datos a memoria o Memory Data Register (MDR)** copia el contenido de esa dirección de memoria a un registro interno de la CPU. Este registro es temporal, un paso antes de que su contenido sea copiado al registro de instrucción.
 - La parte final de la etapa de búsqueda o *fetch* es la escritura de la instrucción en el **registro de instrucción o Instruction Register**, del cual la unidad de control del procesador copiará su contenido para la segunda etapa del ciclo de instrucción.
2. **Decodificación (Decode):** Hay diferentes tipos de instrucciones y no todas hacen lo mismo, por lo que dependiendo del tipo de instrucción necesitamos saber hacia qué unidades de ejecución se van a enviar y la manera más clásica de hacerlo es a través de un decodificador, el cual toma cada instrucción y la divide internamente según el opcode y la dirección de memoria donde se encuentra esta. La **unidad de control (UC)** de la CPU descompone la instrucción y elige la ruta de instrucción correspondiente; como un multiplexor.
3. **Ejecución (Execute):** Es donde las instrucciones son resueltas, pero no todos los tipos de instrucción se resuelven de la misma forma, ya que la forma de utilizar el hardware dependerá del tipo de función de cada una de ellas. En las instrucciones de cálculo, la unidad de control envía los datos a la **unidad aritmético lógica (ALU)**. Si por el contrario se necesita mover información o realizar un salto de control, se llaman a los componentes de enrutamiento de datos y se activan para modificar el flujo del programa.
4. **Almacén de resultados (Write Back/Store):** Consolida el resultado obtenido en la etapa de ejecución para que pueda ser utilizada en el futuro. El destino de almacenamiento depende de la instrucción: puede guardarse en un **registro interno de la CPU** (como un acumulador) para un acceso inmediato, o enviarse de vuelta a una dirección específica de la memoria principal.

Manejo de las Interrupciones

Definición

Una interrupción es una señal generada por un hardware o software cuando un evento necesita la atención inmediata del procesador. Esto causa que el CPU pare temporalmente la ejecución del hilo de instrucción actual, guarda el estado (contexto) y responde con la más alta prioridad la petición saltando a una dirección de memoria donde se encuentra el manejador de interrupciones. En los dispositivos de E/S, las interrupciones son generadas usando la Línea de Petición de Interrupción (*Interrupt Service Line IRQ/IRL*) y son manejadas por una rutina de software llamada Rutina del Servicio de Interrupción (*Interrupt Service Routine ISR*). Luego de la ejecución del ISR, el procesador recupera el programa interrumpido desde donde se pausó.

Maximización de la eficiencia del procesador: Los dispositivos periféricos (como discos duros o teclados) son órdenes de magnitud más lentos que la CPU. Sin interrupciones, el procesador tendría que desperdiciar millones de ciclos de reloj esperando en un bucle cerrado a que un dispositivo termine de leer o escribir datos (*E/S programada o polling*). Las interrupciones permiten que el CPU ejecute otras tareas de usuario mientras el dispositivo trabaja; el hardware avisará activamente solo cuando haya terminado.

Las interrupciones aparecen, principalmente, como una vía para mejorar la eficiencia del procesamiento. Por ejemplo, la mayoría de los dispositivos externos son mucho más lentos que el procesador.

Soporte para la Multiprogramación y Tiempo Compartido: Permiten al sistema operativo arrebatar de forma segura el control de la CPU a un programa (a través de la interrupción del reloj) para dárselo a otro, garantizando que ningún proceso monopolice el sistema.

Manejo asíncrono y seguro de errores: El sistema operativo puede interceptar fallos de software o fallos de hardware en caliente antes de que corrompa el resto del sistema o la memoria RAM.

Tipos de interrupción

Existen dos tipos principales de interrupciones

- **Software:** Estas son generadas por un programa. También son conocidas como trampas o excepciones. Estas interrupciones son utilizadas para solicitar servicios por parte del sistema operativos o para manejar condiciones de error durante la ejecución de un programa.
- **Hardware:**

Tipo de interrupción	Descripción	Origen	Tratamiento
Programa	Generadas por alguna condición que se produce como resultado de la ejecución de una instrucción.	- Desbordamiento aritmético. - División por cero. - Intento de ejecutar una instrucción ilegal de la máquina. - Referencia a una zona de memoria fuera del espacio permitido al usuario.	El procesador suspende el programa del usuario e invoca al núcleo del SO. Si el error es una falla de página (<i>page fault</i>), el SO carga la página requerida en la RAM y reanuda la instrucción. Si es un error fatal (ej. división por cero), el SO aborta el proceso, libera su memoria y genera un reporte de error.
Reloj	Permite al sistema operativo llevar a cabo ciertas funciones con determinada regularidad.	- Reloj o temporizador interno del hardware del procesador.	El procesador cede el control al planificador (<i>scheduler</i>) del sistema operativo. El SO actualiza la hora del sistema, verifica si el proceso actual ha agotado su rodaja de tiempo (<i>quantum</i>) y, de ser así, realiza un cambio de contexto para poner a ejecutar el siguiente proceso de la cola.
E/S	Indicar que una operación ha terminado normalmente o para indicar diversas condiciones de error.	Generadas por un controlador de E/S: - Disco duro - Tarjeta de red - Teclado - Etc.	El procesador interrumpe la tarea actual y ejecuta el <i>driver</i> del dispositivo. Este gestiona los datos, cambia el estado del proceso que solicitó la E/S de "Bloqueado" a "Listo" para que vuelva a competir por la CPU, y limpia la señal física de interrupción.
Fallo del Hardware	Generada por una avería física.	- Corte o bajada en el suministro eléctrico - Error de paridad de la memoria RAM o caché.	El sistema operativo suspende de inmediato toda ejecución normal para proteger los datos almacenados. Si es un fallo de energía y los condensadores dan unos milisegundos de margen, el SO realiza un guardado de emergencia de los registros de sistema. Si es un fallo de memoria, el SO detiene la máquina por completo (Kernel Panic o Pantalla Azul) para evitar que datos corruptos se escriban en el disco.

Enmascaramiento de interrupciones

El enmascaramiento es la capacidad que tiene el procesador para **ignorar o postergar** temporalmente ciertas señales de interrupción. Esto se logra modificando bits específicos en un registro de control del procesador.

Dependiendo de si se puede ignorar o no, las interrupciones se dividen en dos categorías:

- **Interrupciones Enmascarables:** Son aquellas que el procesador puede decidir "bloquear" o poner en espera. Si la máscara está activa para una interrupción concreta, el hardware periférico puede enviar la señal, pero el procesador la ignorará momentáneamente y continuará ejecutando las instrucciones actuales. Se reactivan cuando el código crítico termina.
- **Interrupciones No Enmascarables (NMI - Non-Maskable Interrupts):** Son líneas de interrupción de altísima prioridad que el procesador **jamás** puede ignorar ni bloquear. Están reservadas para eventos catastróficos que ponen en peligro la integridad del sistema, como un fallo crítico en la memoria RAM o una pérdida inminente de energía eléctrica.

El uso principal del enmascaramiento es garantizar la **atomicidad** en secciones críticas del código. Si el sistema operativo está actualizando una estructura de datos vital (como la lista de procesos listos), una interrupción en ese preciso milisegundo podría corromper la información. El SO enmascara las interrupciones antes de entrar a esa sección y las desenmascara al salir.

Esquemas de prioridad de interrupción

Los esquemas de prioridad de interrupciones se utilizan en microprocesadores y microcontroladores para gestionar múltiples solicitudes de interrupción (IRQ). Estos esquemas aseguran que las tareas más urgentes se procesen antes que las menos importantes, lo que los hace esenciales para los sistemas en tiempo real y la gestión eficiente de interrupciones.

Tipos de esquemas de prioridad de interrupción

- **Esquema de prioridad fija:** Cada interrupción tiene un nivel de prioridad predeterminado. La interrupción con mayor prioridad se gestiona primero. Si dos interrupciones ocurren simultáneamente, se atiende la que tiene mayor **prioridad** "La interrupción A, con prioridad 1, se atiende antes que la interrupción B, con prioridad 2".
- **Esquema de prioridad dinámica:** La prioridad de una interrupción puede cambiar según las condiciones del sistema. Esto ayuda a priorizar tareas en tiempo real o críticas sobre otras. Por ejemplo, un sistema puede aumentar la prioridad de una interrupción de sensor en función de su importancia en un momento determinado.
- **Esquemas de interrupciones vectorizadas:** Aquí, cada interrupción tiene una dirección de memoria específica (vector). El procesador salta a esta dirección para gestionar la interrupción, y la interrupción con mayor prioridad se procesa **primero**. Por ejemplo, un sistema que utilice direcciones de memoria para gestionar rápidamente las interrupciones más urgentes.
- **Enmascaramiento de prioridad:** Este esquema permite desactivar temporalmente las interrupciones de menor prioridad, asegurando que las interrupciones de alta prioridad se gestionen de inmediato y sin demora. "Si la interrupción A es más crítica que la interrupción B, la interrupción B puede enmascarse hasta que se procese la interrupción A".
- **Esquema de prioridad round-robin:** Las interrupciones se procesan en orden cíclico, asegurando que cada interrupción se maneje de forma justa, especialmente cuando todas las interrupciones tienen la misma prioridad. "Las interrupciones A, B y C se manejan de forma de todos contra todos".

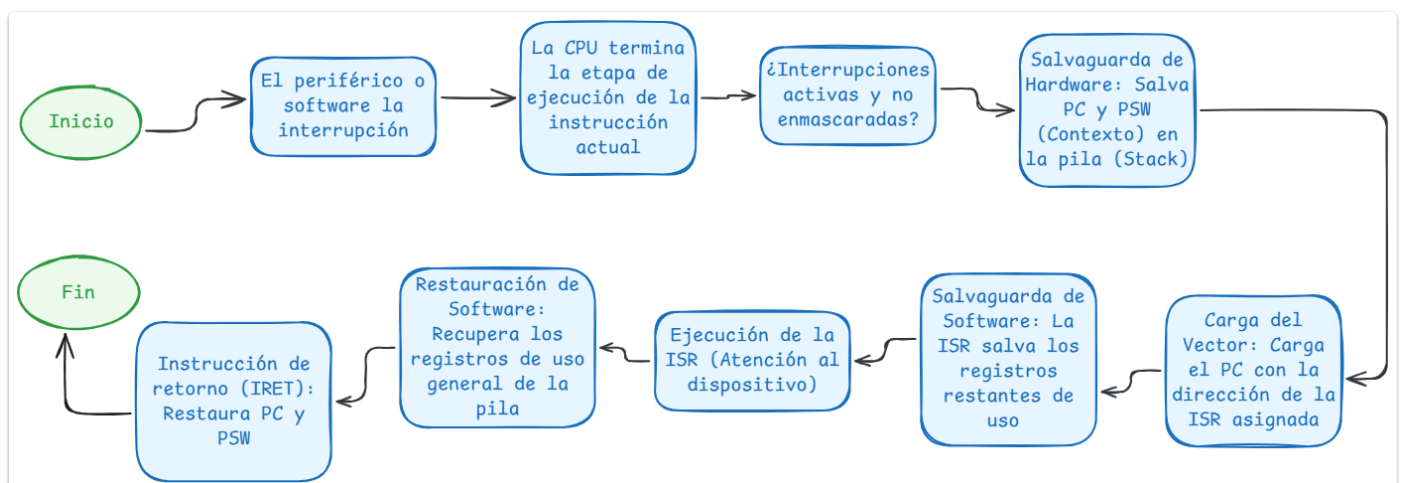
¿Qué pasa si ocurren dos interrupciones al mismo tiempo y del mismo tipo?

Si hay dos interrupciones al mismo tiempo y son del mismo tipo, sean dos E/S, cada una viaja en canales distintos hacia el **Controlador Programable de Interrupciones (PIC o APIC)**. Dentro, el controlador desempata con reglas preestablecidas por fábrica, el controlador evalúa ambas señales en el mismo ciclo de reloj, selecciona la que tenga la prioridad física más alta y se la envía a la CPU.

La interrupción que pierde el desempate se queda en un registro interno en estado **pendiente**.

El procesador escoge la interrupción ganadora, guarda el contexto del programa del usuario y ejecuta su respectiva ISR. Al finalizar esa rutina el procesador comprueba si hay interrupciones pendientes antes de regresar al programa del usuario. Al detectar la segunda interrupción retenida, la CPU salta inmediatamente a ejecutar su ISR de manera **secuencial**.

Gestión de una interrupción



- **Paso 1:** Cada vez que se genera una interrupción, puede ser una interrupción de E/S o una interrupción del sistema.

- **Paso 2:** El estado actual que comprende los registros y el contador de programa se almacena para conservar el estado del proceso.
- **Paso 3:** La interrupción de corriente y su manejador se identifican a través de la tabla de vectores de interrupciones en el procesador.
- **Paso 4:** Este control ahora para al gestor de interrupciones, que es una función ubicada en el espacio del núcleo.
- **Paso 5:** Las tareas específicas se realizan mediante la ISR, que son esenciales para gestionar la interrupción.
- **Paso 6:** Se recupera el estado de la sesión anterior para construir sobre el proceso desde ese punto.
- **Paso 7:** El control se traslada de nuevo al otro proceso que estaba pendiente y el proceso normal continúa.

Estructura del Kernel de Linux

Última versión del kernel de Linux (6.xx)

La nueva versión del Kernel de Linux 7.0 fue lanzado oficialmente el 12 de abril de 2026. Aunque deja atrás la serie 6.x, el cambio de nomenclatura no se debe a una reescritura total del sistema, sino una cuestión de simplicidad cronológica para evitar que los números menores en la versión sean muy altos. "El salto del 6.x al 7.0 es simbólico".

Sin embargo, algunos cambios estructurales que implementa esta nueva versión son los siguientes:

1. **Simplificación del modelo de preempción (Preemption Selection):** Con el fin de limpiar el código base del planificador de tareas (*scheduler*) y simplificar la arquitectura del sistema, Linux 7.0 reduce las opciones de preempción. Para las arquitecturas modernas más utilizadas (como **x86**, **arm64**, **riscv**, **powerpc**, **loongarch** y **s390**), el núcleo ahora se limita estructuralmente a solo dos modelos: **Full** (preempción completa) y **Lazy** (Preempción perezosa). Esto elimina la fragmentación de código que arrastraba la serie 6.x en entornos de alta exigencia y sistemas embebidos.

La preempción es la capacidad de un sistema operativo o planificador para interrumpir temporalmente una tarea en ejecución, sin su cooperación, para ceder los recursos a otra tarea de mayor prioridad. Garantiza multitarea.

Un sistema embebido es un sistema de computación diseñado para realizar una o varias funciones específicas, a diferencia de una computadora de uso general. Combina hardware y software integrados directamente en un dispositivo.

2. **Rust pasa a ser un componente permanente:** En las últimas versiones de la serie 6.x, la integración del lenguaje Rust en el núcleo se mantenía como un ecosistema experimental. Con la llegada de Linux 7.0 **la infraestructura de Rust se consolida de forma permanente en la arquitectura** del núcleo. Esto altera la estructura de desarrollo permitiendo que los nuevos controladores (*drivers*) y módulos nativos se beneficien de la seguridad en memoria integrada de Rust directamente en la línea principal (*mainline*) sin depender de características experimentales.
3. **Verificación estática de seguridad en bloqueos (Clang Lock Checking):** Para evitar condiciones de carrera (*race conditions*) y bloqueos mutuos que afecten la estabilidad estructural del sistema, Linux 7.0 añade soporte formal para extensiones del compilador Clang. Estas extensiones permiten realizar un **análisis estático en tiempo de compilación** para comprobar de manera estricta si determinados contextos de bloqueo (*context locks*) están activos o inactivos. Esto eleva la seguridad interna del código sin añadir sobrecarga de rendimiento al procesador en tiempo de ejecución.

Clang es un compilador de código abierto para lenguaje de la familia de C. Diseñado como reemplazo directo y rápido de GCC. Destaca por ofrecer mensajes de error y advertencia muy precisos y fáciles de entender.

4. **Nueva capa de indirección en sistemas de archivos (Btrfs y XFS):**
 - **Btrfs:** Introduce de forma experimental el **árbol de remapeo lógico** (*remap tree*). Estructuralmente actúa como una capa de indirección para las operaciones de E/S. Cuando se reubican bloques de datos, en lugar de corregir minuciosamente cada estructura de árbol en el disco (como se hacía en la serie 6.x), el sistema simplemente registra la antigua y la nueva dirección en el árbol de remapeo mejorando la flexibilidad y la fiabilidad.
 - **XFS:** Implementa una arquitectura interna de **monitoreo de salud en vivo**. Mediante la creación de archivos anónimos, el kernel notifica eventos de salud del sistema de archivos directamente al espacio de usuario (*user space*). Esto permite reparar metadatos corruptos de forma automática en caliente sin bloquear los procesos de desmontaje de discos.

Los metadatos son, literalmente "datos sobre datos". Proporcionan información adicional sobre el origen, el contexto, la estructura y el uso de un archivo, imagen o documento

5. **Optimización estructural del subsistema de Swapping:** La gestión de memoria virtual ha sufrido una modificación muy esperada en escenarios de baja memoria (como dispositivos móviles o PCs con poca RAM que usan **zram**). En las versiones 6.x el kernel requería

descomprimir las páginas de memoria de la `zram` antes de poder volcarlas en el disco físico. Linux 7.0 reestructura este flujo: ahora **puede escribir los datos comprimidos de zram directamente en el dispositivo de almacenamiento de respaldo** sin pasar por la fase intermedia de descompresión de la RAM, ahorrando ciclos de CPU y reduciendo drásticamente la latencia bajo cargas masivas de trabajo.

Beneficios de Rust en el Kernel de Linux

La inclusión de Rust como segundo lenguaje oficial en el Kernel de Linux (junto a C) representa uno de los cambios más disruptivos e importantes en la historia del proyecto. Tradicionalmente, los sistemas operativos se escriben en C porque este lenguaje ofrece un control absoluto sobre el hardware y la memoria, pero traslada toda la responsabilidad de la seguridad al programador.

Rust resuelve este dilema histórico proporcionando **seguridad de memoria en tiempo de compilación sin sacrificar el rendimiento**.

Beneficios principales

Ownership & Borrowing (Propiedad y Préstamo)

En el desarrollo de un kernel en C, la gestión de memoria es manual (`malloc()` y `free()`). Esto da origen a los errores más peligrosos y difíciles de depurar en un sistema operativo:

- *Use-After-Free*: usar memoria que ya fue liberada.
- *Double Free*: liberar la misma memoria dos veces.
- *Memory Leaks*: fugas de memoria.

En el espacio del kernel, estos errores causan fallos críticos de seguridad o inestabilidad total.

Rust introduce el concepto de **Ownership**: cada porción de memoria o recurso en el código tiene un único "dueño" (una variable). Cuando ese dueño sale de su ámbito de ejecución (*scope*), la memoria se libera automáticamente de forma determinista. El **Borrowing** permite que otros componentes "pidan prestada" esa memoria mediante referencias, aplicando una regla estricta: se pueden tener infinitas referencias de solo lectura o una sola referencia mutable (modificable) a la vez, pero nunca ambas.

El *Borrow Checker* (el verificador de préstamos del compilador) analiza esto antes de generar el binario. Si un desarrollador intenta escribir un driver que libere la memoria antes de que un hilo termine de usarla, **el código simplemente no compila**. Además, la regla de la referencia mutable única elimina por completo las **Data Races** (condiciones de carrera de datos) en entornos multiprocesador, garantizando la concurrencia segura a nivel de núcleo.

Sin Null Pointers (Ausencia de punteros nulos)

En C, los punteros son omnipresentes. Es extremadamente común que una función devuelva un puntero `NULL` para indicar un error, la ausencia de un dispositivo o que un recurso no está listo. Si un programador olvida verificar si el puntero es `NULL` antes de intentar acceder a su contenido (desreferenciación), el procesador genera una falla de hardware que en el espacio de usuario cierra una aplicación, pero en el espacio de núcleo provoca un **Kernel Panic** (caída completa del sistema).

En el código seguro de Rust, los punteros nulos tradicionales no existen. Para representar la posibilidad de que un valor no exista, Rust utiliza un tipo de dato llamado `Option<T>`, que es una enumeración con dos variantes:

- `Some(valor)` si el dato existe.
- `None` si no existe

El compilador de Rust obliga explícitamente al desarrollador a controlar ambos escenarios (`Some` y `None`) mediante estructuras de control como `match`. No es posible acceder accidentalmente al contenido de un valor que no existe. Esto erradica por completo los fallos por desreferenciación de punteros nulos en los módulos de Rust, haciendo que los nuevos drivers de hardware sean drásticamente más estables y robustos frente a imprevistos.

Abstracciones de Costo Cero (Zero-Cost Abstractions)

El diseño del Kernel de Linux prioriza el rendimiento por encima de todo. Un sistema operativo no puede permitirse el lujo de usar lenguajes modernos como Java, Python o Go porque estos dependen de un *Garbage Collector* (recolector de basura) o de entornos de ejecución (*runtime*) que consumen ciclos de CPU y pausan el sistema de forma impredecible para limpiar memoria. Por ello, en C se suele programar a muy bajo nivel, lo que a veces sacrifica la legibilidad y la reutilización del código en pos de la velocidad.

El concepto de "abstracción de costo cero" (heredado de C++) estipula que aquello que no usas, no lo pagas; y el código de alto nivel que sí usas, se compila en un código máquina tan óptimo como si lo hubieras escrito minuciosamente a mano a bajo nivel. Rust permite usar conceptos avanzados y legibles -como iteradores funcionales, genéricos, tipado complejo, cierres (*closures*) y polimorfismo- que desaparecen durante la compilación.

Rust permite a los ingenieros del kernel escribir interfaces de programación (*APIs*) modernas, limpias, seguras y reutilizables para gestionar el hardware, teniendo la certeza absoluta de que el compilador traducirá todo eso a instrucciones de ensamblador ultra eficientes. Rust compila directamente el código nativo, no tiene runtime ni recolector de basura, cumpliendo con las estrictas exigencias de latencia y rendimiento que requiere el núcleo de Linux para competir directamente con el C tradicional.

Virtualización

Definición

La **virtualización** es una tecnología de software que permite crear entornos virtuales (simulados) a partir de un único sistema de hardware físico. Mediante una capa intermedia de software (comúnmente llamada hipervisor o monitor de máquina virtual), se abstraen los recursos físicos -como el procesador, la memoria, el almacenamiento y la red- para dividirlos en múltiples recursos lógicos e independientes, permitiendo ejecutar varios sistemas operativos o aplicaciones en la misma máquina física de forma simultánea y aislada.

Tipos de Virtualización

Virtualización de Hardware (o de Servidor)

Consiste en abstraer los componentes físicos de un servidor para crear máquinas virtuales (*MV*) independientes, cada una con su propio sistema operativo ("sistema operativo invitado").

- **Virtualización Completa (*Full Virtualization*)**: El hipervisor emula por completo el hardware subyacente. El sistema operativo invitado no sabe que está virtualizado y no requiere modificaciones.
- **Paravirtualización**: El sistema operativo invitado es modificado de forma deliberada antes de instalarse para que sea consciente de que está virtualizando. Se comunica con el hipervisor a través de llamadas especiales (*hypercalls*), lo que mejora el rendimiento.
- **Virtualización Asistida por Hardware**: Utiliza extensiones físicas integradas directamente en los procesadores modernos (como Intel VT-x o AMD-V) para optimizar y acelerar las tareas de virtualización de la CPU.

Ventajas	Desventajas
Maximiza la consolidación de servidores y reduce costos de hardware y energía.	Sobrecarga de rendimiento (<i>overhead</i>) debido a la capa de hipervisor que traduce las instrucciones de hardware.
Aislamiento total de seguridad entre las diferentes máquinas virtuales.	Alto consumo de recursos (cada MV requiere duplicar memoria RAM y espacio en disco para su propio kernel de sistema operativo).
Facilidad para realizar copias de seguridad de servidores completos y migraciones en caliente (<i>live migration</i>).	

Virtualización de Software (o de Sistema Operativo y Aplicaciones)

Es la abstracción que permite aislar entornos de ejecución de software o aplicaciones individuales del sistema operativo anfitrión, o bien, crear contenedores aislados que comparten el mismo núcleo.

- **Virtualización a Nivel de Sistema Operativo**: El kernel de un único sistema operativo anfitrión se divide en múltiples instancias aisladas en el espacio de usuario (conocidos como *contenedores*).
- **Virtualización de Aplicaciones**: El software se encapsula y se ejecuta de forma aislada dentro del sistema operativo, portando sus propias librerías y configuraciones (por ejemplo, VMware ThinApp), evitando conflictos de dependencias con otras apps.

Ventajas	Desventajas
Consumo de recursos extremadamente bajo comparado con la virtualización de hardware.	Dependencias del kernel: Todos los entornos virtuales deben compartir el mismo tipo de sistema operativo base (por ejemplos, contenedores Linux solo corren en Linux)

Ventajas	Desventajas
Inicio de aplicaciones o entornos casi instantáneo.	Menor aislamiento de seguridad: si el kernel del sistema operativo anfitrión se ve comprometido, todos los entornos virtuales de software sufren el mismo riesgo
Elimina por completo el problema de "en mi máquina si funciona" al empaquetar todas las dependencias.	

Virtualización de Almacenamiento

Consiste en agrupar múltiples dispositivos de almacenamiento físico (discos duros individuales, arreglos de discos, sistemas SAN/NAS) provenientes de diferentes proveedores para presentarlos como un único bloque lógico de almacenamiento centralizado.

- **Virtualización basada en Bloques:** Abstrae el acceso físico a los discos (SAN). El sistema operativo ve un volumen o disco lógico único gigante, ocultando la complejidad de donde están distribuidos físicamente los sectores.
- **Virtualización basada en Archivos (NAS):** Optimiza el acceso a sistemas de archivos distribuidos en red (como NFS o SMB), rompiendo la dependencia entre los datos compartidos y el servidor físico específico que los aloja.

Ventajas	Desventajas
Gestión centralizada y simplifica de todo el almacenamiento de una empresa.	Si la capa de virtualización de almacenamiento falla o se satura, se puede perder el acceso a los datos de toda la infraestructura.
Facilita funciones avanzadas como el aprovisionamiento dinámico (<i>thin provisioning</i>) y la replicación de datos sin detener servicios.	Requiere configuraciones complejas y arquitecturas de red dedicadas de alta velocidad (como redes de Fibra Óptica).

Virtualización de Datos

Es el enfoque que permite integrar datos provenientes de múltiples fuentes heterogéneas (bases de datos SQL, NoSQL, Excel, servicios en la nube) en una **capa de datos única y virtual**, sin necesidad de mover, copiar o replicar físicamente los datos originales.

- **Federación de Datos:** Técnica que toma consultas en tiempo real y las distribuye analíticamente entre las diferentes fuentes físicas, devolviendo un resultado unificado al usuario.
- **Servicios de Datos (Data as a Service - Daas):** Transforma la información de las fuentes subyacentes y la expone formalmente a través de APIs de software (como servicios REST) para consumo inmediato.

Ventajas	Desventajas
Acceso a la información en tiempo real sin retrasos por procesos de carga masiva (ETL).	El rendimiento depende directamente de la velocidad de red y del estado de carga de los sistemas de origen.
Reduce drásticamente los costos de almacenamiento al no duplicar la información.	El diseño de consultas complejas sobre fuentes muy diferentes puede sobrecargar los servidores operativos si no se optimiza de forma estricta.
Oculta la complejidad técnica y los formatos de las bases de datos originales a los analistas de negocio.	

Virtualización de Red

Cosiste en desacoplar los servicios de red del hardware físico subyacente (switches, routers, cables de red) para crear redes lógicas independientes, programables y altamente flexibles sobre la misma infraestructura física.

- **Virtualización Externa (VLANs/Redes Virtuales):** Divide una red física local en múltiples dominios de difusión lógicos independientes utilizando etiquetas en las tramas de red.
- **Virtualización Interna (Redes Definidas por Software - SDN / Encapsulamiento):** Desconecta el plano de control (las decisiones de hacia dónde van los datos) del plano de datos (el hardware que mueve los bits). Utiliza protocolos como VXLAN o NVGRE para crear redes lógicas complejas superpuestas (*overlay networks*).

Ventajas	Desventajas
Permite crear, modificar o destruir topologías de red completas (firewalls, routers, balanceadores) mediante software en cuestión de segundos.	Introduce una gran complejidad en el diagnostico de fallas (hacer un <i>troubleshooting</i> se vuelve difícil porque la topología física no coincide con la lógica).
Aislamiento y segmentación de seguridad estricta para infraestructuras multi-inquilino (<i>multi-tenant</i>).	Mayor consumo de procesamiento en la tarjetas de red debido al proceso constante de encapsulado y desencapsulado de paquetes.

Contenedores vs Hipervisores

Para comprender la diferencia fundamental, los **hipervisores** se enfocan en virtualizar el **hardware** (creando máquinas virtuales completas e independientes), mientras que los **contenedores** se enfocan en virtualizar el **sistema operativo** (compartiendo el mismo núcleo y aislando únicamente las aplicaciones).

Característica	Hipervisores (Máquinas Virtuales)	Contenedores (Docker)
Nivel de Abstracción	Hardware físico (CPU, Memoria, Discos, Red).	Sistema Operativo (Espacio de usuario).
Aislamiento	Muy Alto. Cada MV corre en su propio kernel aislado; un fallo en una MV no afecta a las demás.	Moderado. Comparten el mismo Kernel del host; un fallo crítico en el kernel puede afectar a todos los contenedores
Consumo de Recursos	Alto. Requieren gigabytes de RAM y almacenamiento para alojar el sistema operativo completo por cada MV	Milisegundos o fracciones de segundo (es simplemente iniciar un proceso nativo en el host).
Tiempo de Arranque	Minutos o segundos largos (debe iniciar todo el proceso de boot del sistema operativo invitado)	Milisegundos o fracciones de segundo (es simplemente iniciar un proceso nativo en el host).
Portabilidad	Limitada por la compatibilidad de formatos del hipervisor (VMDK, VHDX) y la arquitectura del hardware.	Muy Alta. Un contenedor se ejecuta exactamente igual en cualquier máquina que tenga instalado el motor de contenedores.
Casos de Uso Ideales	Consolidación de servidores físicos heterogéneos, entornos que requieren sistemas operativos diferentes (ej. Windows sobre Linux) o máxima seguridad.	Arquitecturas de Microservicios, metodologías de desarrollo ágil (CI/CD), despliegue rápido de aplicaciones escalables.

Referencias Bibliográficas

Organization of Computer Systems:

William Stallings. Sistemas Operativos, segunda edición.

M.S. Schmalz. Appendix B: Overview of Computer Organization. [Organization of Computers: Overview](#)

Interrupt In Operating System. [Interrupt In Operating System - GeeksforGeeks](#)